CSE 270 Sports Analytics

Introduction to R

Habet Madoyan

Intro to R

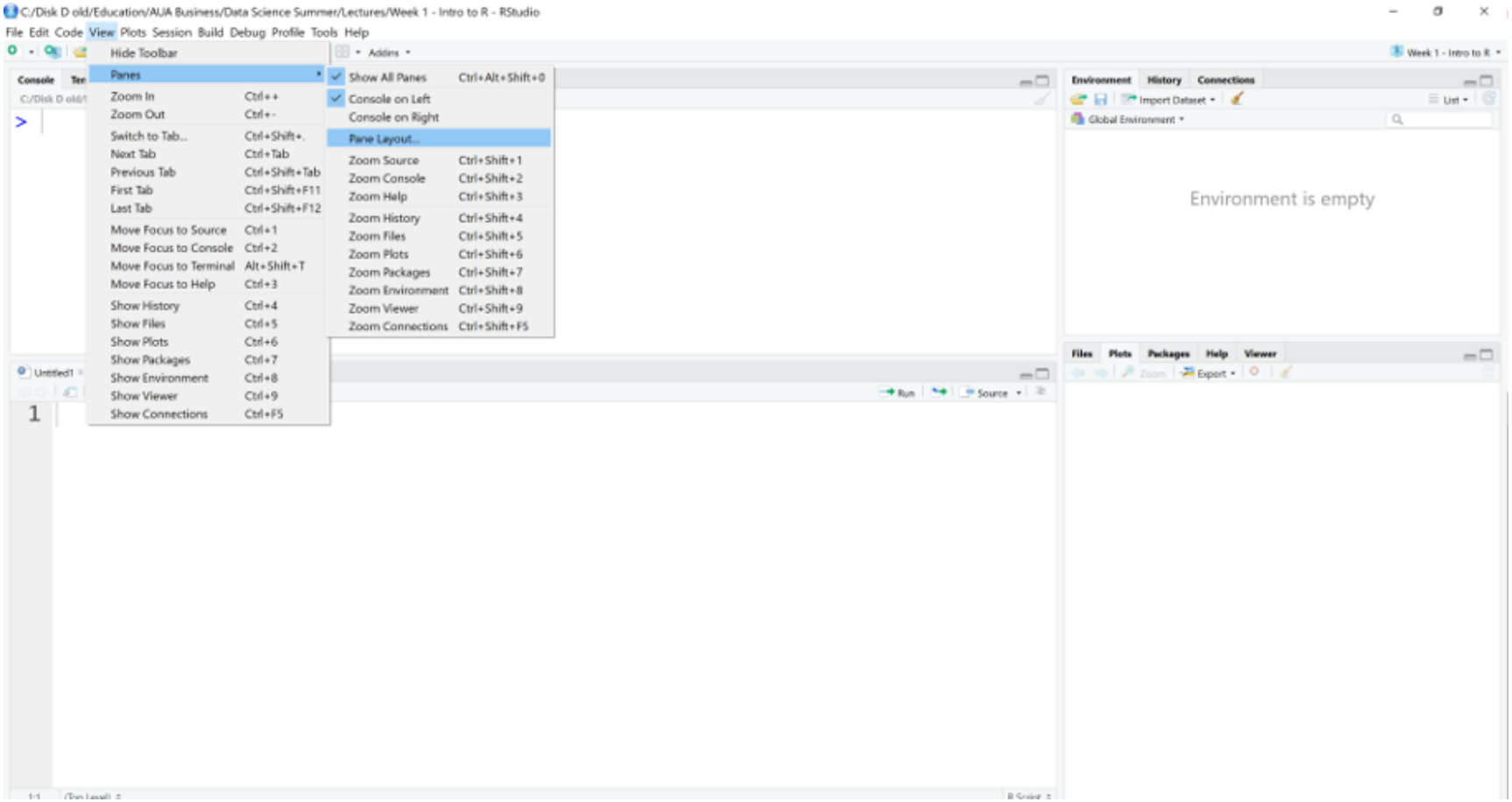
Intro to R

- Go to Comprehensive R Archive Network (CRAN) and install version for your operating system
- Go to RStudio and install RStudio IDE
- You need to install R before installing RStudio

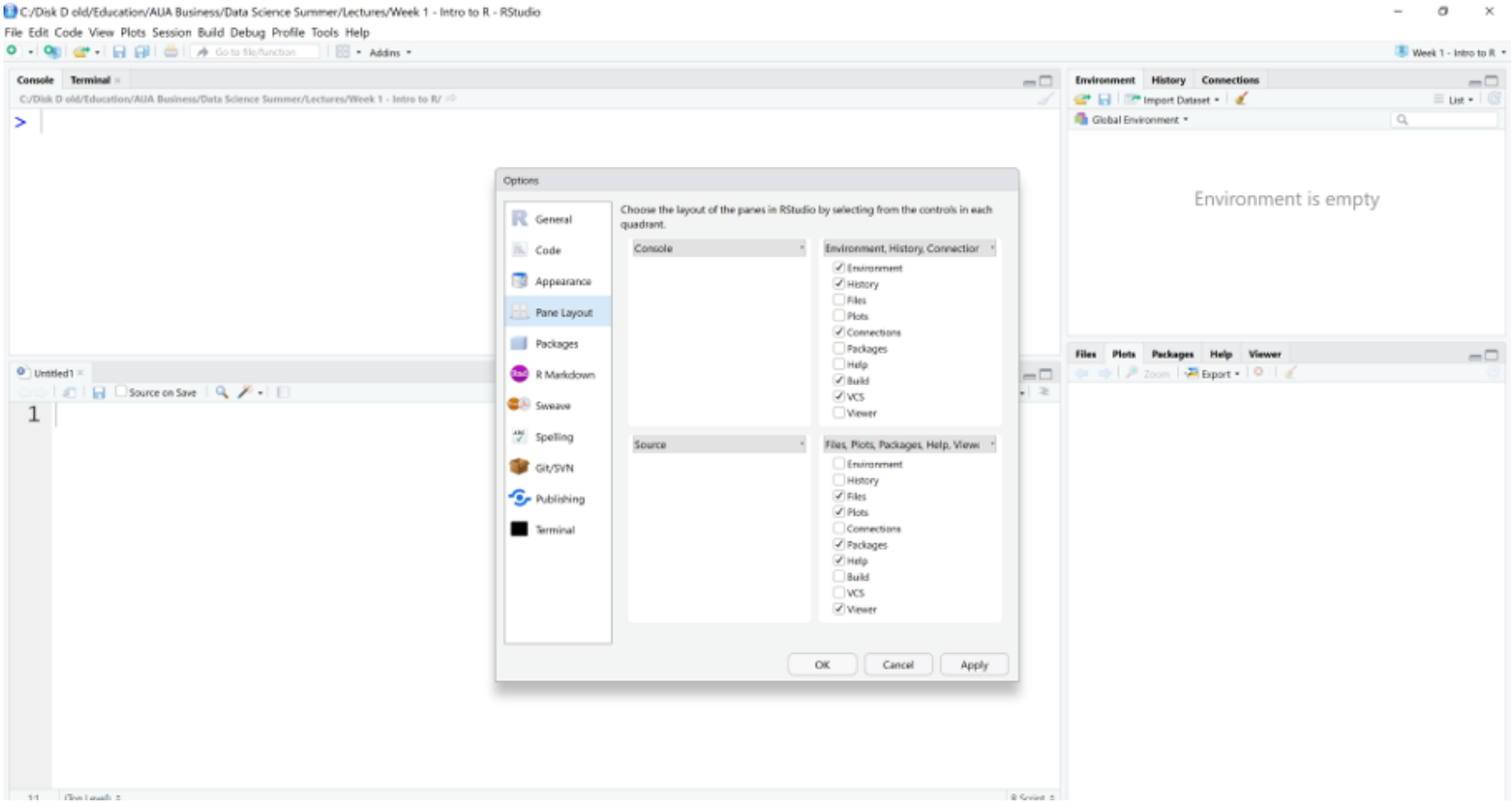


Intro to R

Configure the pane layout of RStudio as you wish



Intro to R


C:/Disk D old/Education/AUA Business/Data Science Summer/Lectures/Week 1 - Intro to R - RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Console
Terminal
Environment
History
Connections
Environment is empty
Files
Plots
Packages
Help
Viewer
Untitled1
Source on Save
Options
General
Code
Appearance
Pane Layout
Packages
R Markdown
Sweave
Spelling
Git/SVN
Publishing
Terminal
Choose the layout of the panes in RStudio by selecting from the controls in each quadrant.
Console
Environment, History, Connection
Source
Files, Plots, Packages, Help, Viewe
OK
Cancel
Apply

Intro to R

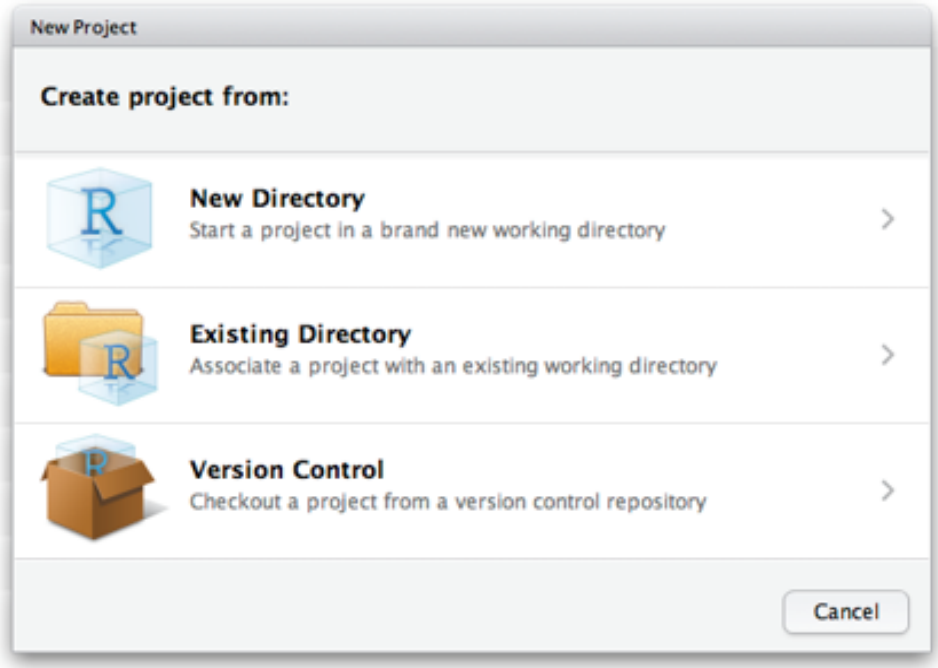
RStudio projects make it straightforward to divide your work into multiple contexts, each with their own working directory, workspace, history, and source documents.

Creating Projects

- RStudio projects are associated with R working directories. You can create an RStudio project:
  - in a brand new directory;
  - in an existing directory where you already have R code and data;
  - by cloning a version control (Git or Subversion) repository.

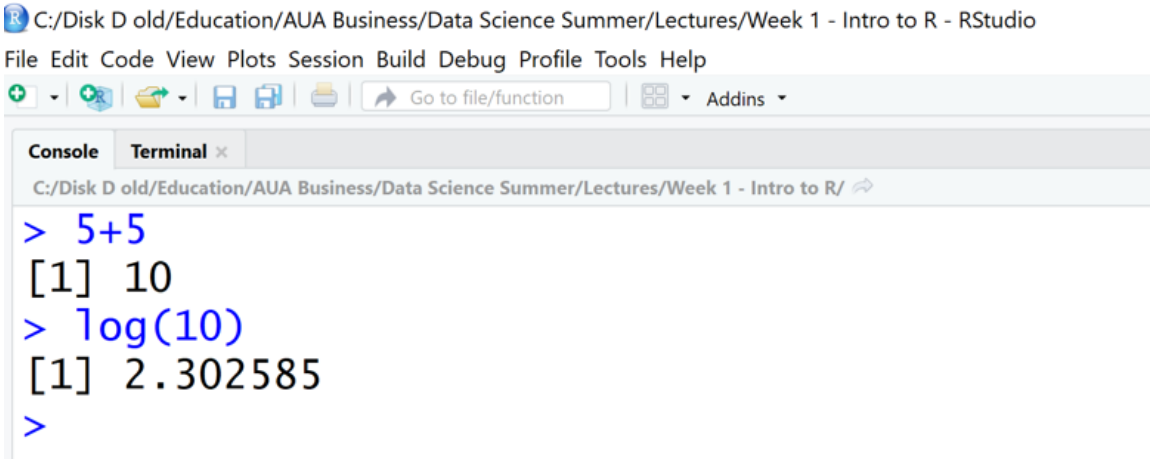
Intro to R

- Create an empty directory
- Go to File → New Project
- Choose existing directory



Intro to R

- Now when you have created your project, all files associated with the project need to be saved in the project directory
- To check the project directory do getwd() in Console



The name of the project appears at the top of RStudio window

Why do we need RStudio Project?

The benefits of RStudio project:

- Can have different projects with their own environment.
- While opening a project, R restores previous work and project history includes recent commands which had been used in the project.
- Source pane remembers the files which had been opened.

Intro to R

When a new project is created, RStudio:

- Creates a project file (with an .Rproj extension) within the project directory. This file contains various project options (discussed below) and can also be used as a shortcut for opening the project directly from the filesystem.
- Creates a hidden directory (named .Rproj.user) where project-specific temporary files (e.g. auto-saved source documents, window-state, etc.) are stored. This directory is also automatically added to .Rbuildignore, .gitignore, etc. if required.
- Loads the project into RStudio and displays its name in the Projects toolbar (which is located on the far right side of the main toolbar).

Intro to R

Opening Projects

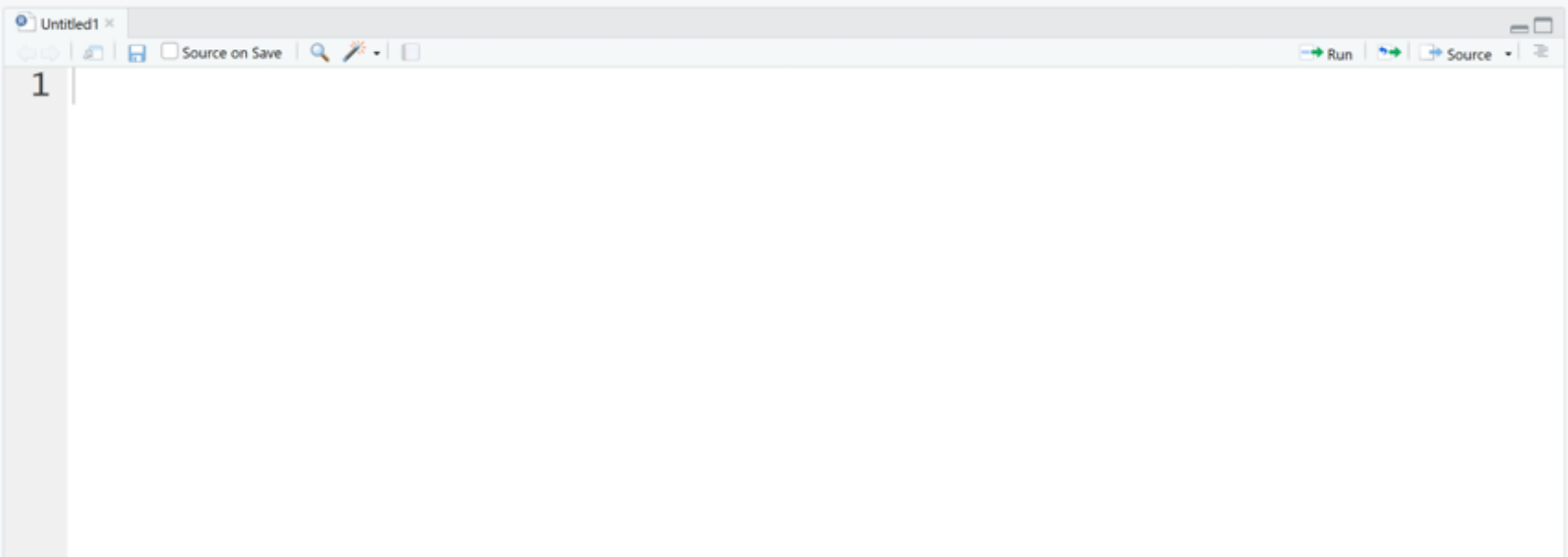
There are several ways to open a project:

- Using the **Open Project** command (available from both the Projects menu and the Projects toolbar) to browse for and select an existing project file (e.g. MyProject.Rproj).
- Selecting a project from the list of the most recently opened projects (also available from both the Projects menu and toolbar).
- Double-clicking on the project file within the system shell (e.g. Windows Explorer, OSX Finder, etc.).

Intro to R

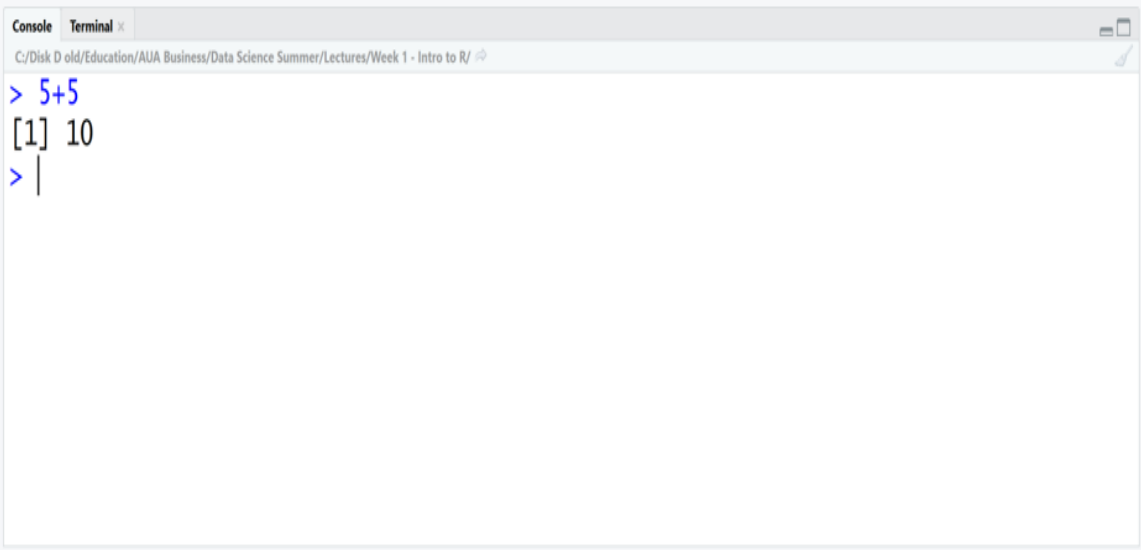
The source editor

- If you plan to reuse your code, write it in source editor



Intro to R

Here you can type any valid R command after the $>$ prompt followed by **Enter** and R will execute that command.

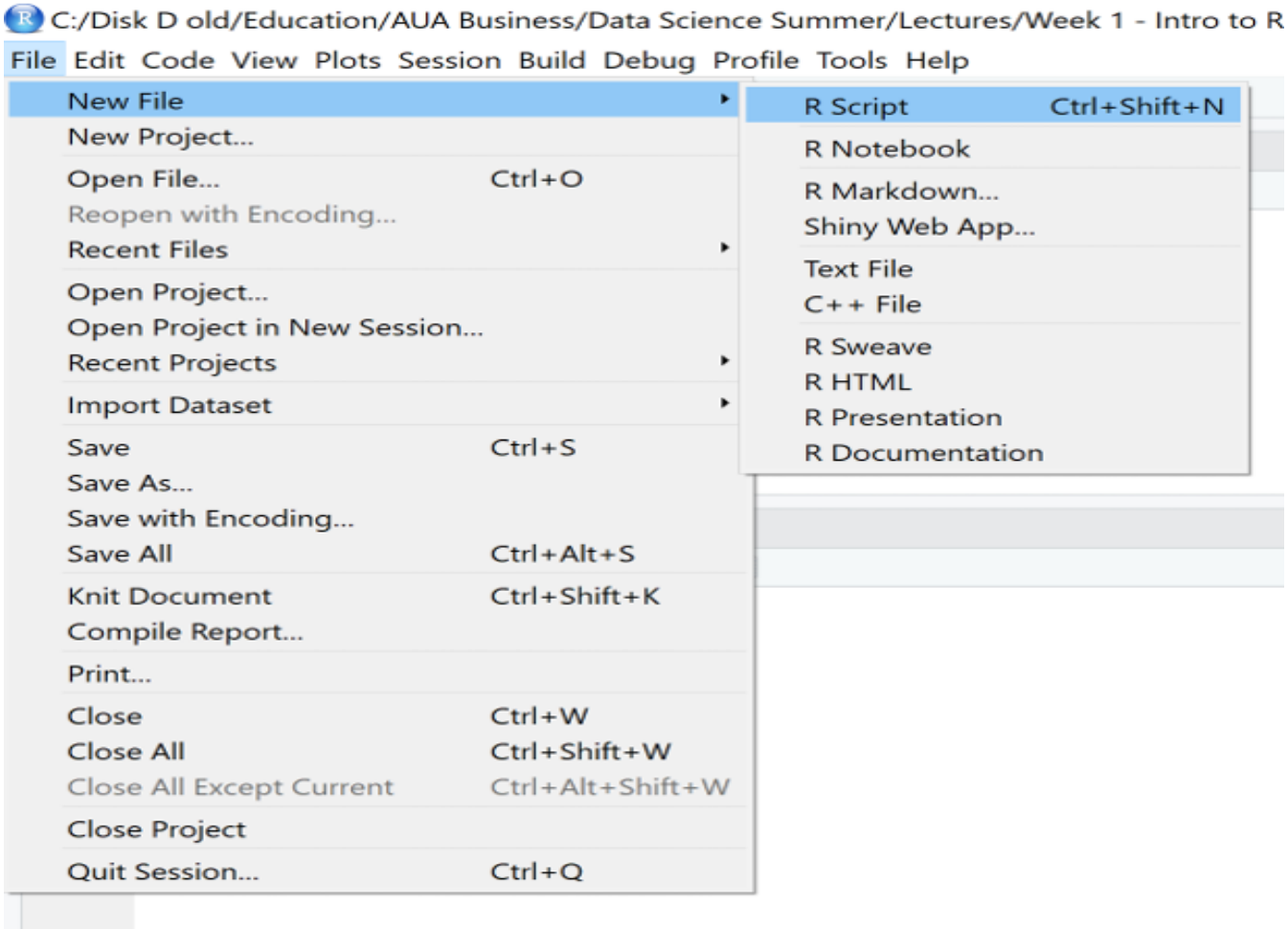


Use console as a calculator

[1] is the index for the output, just ignore it

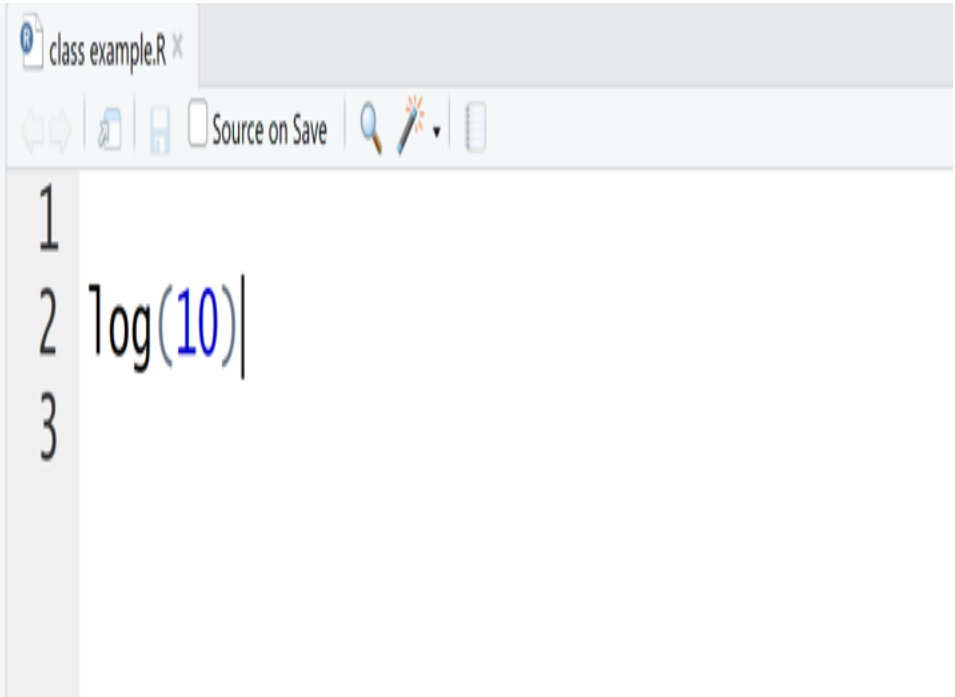
Intro to R

This will create new script file



Intro to R

- To run the script line from source editor, put the cursor anywhere on the line and hit **Ctrl+Enter**
- You will see the output in the console



Intro to R

- If you have a piece of text in your editor that is not a code (thus is not executable by R) then you need to comment it, add # before each line
- When you run this line of code, it will be printed in Console as a text
- To comment large chunk of text, use **Ctrl+Shift+C**

```
C:/Disk D old/Education/AUA Business/Data Science Summer/Lectures/Week 1 - Intro to R/
> # This command will calculate the logarithm of 10
>
> log(10)
[1] 2.302585
>
> # This line will calculate factorial of 10
>
> factorial(10)
[1] 3628800
>
```

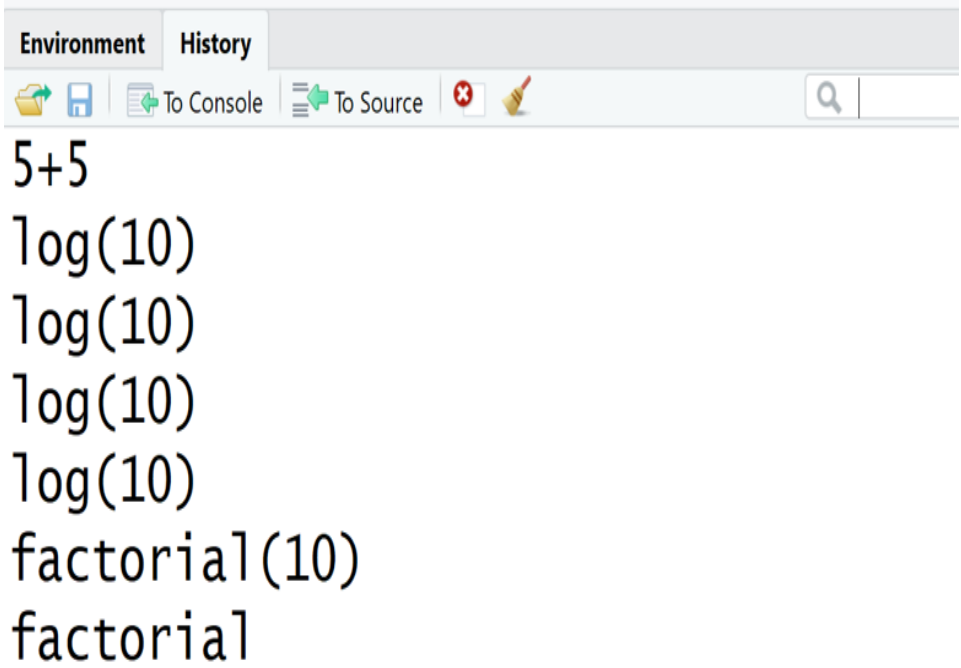
class example.R* ×

Source on Save

```
1
2 # This command will calculate the logarithm of 10
3
4 log(10)
5
6 # This line will calculate factorial of 10
7
8 factorial(10)
```

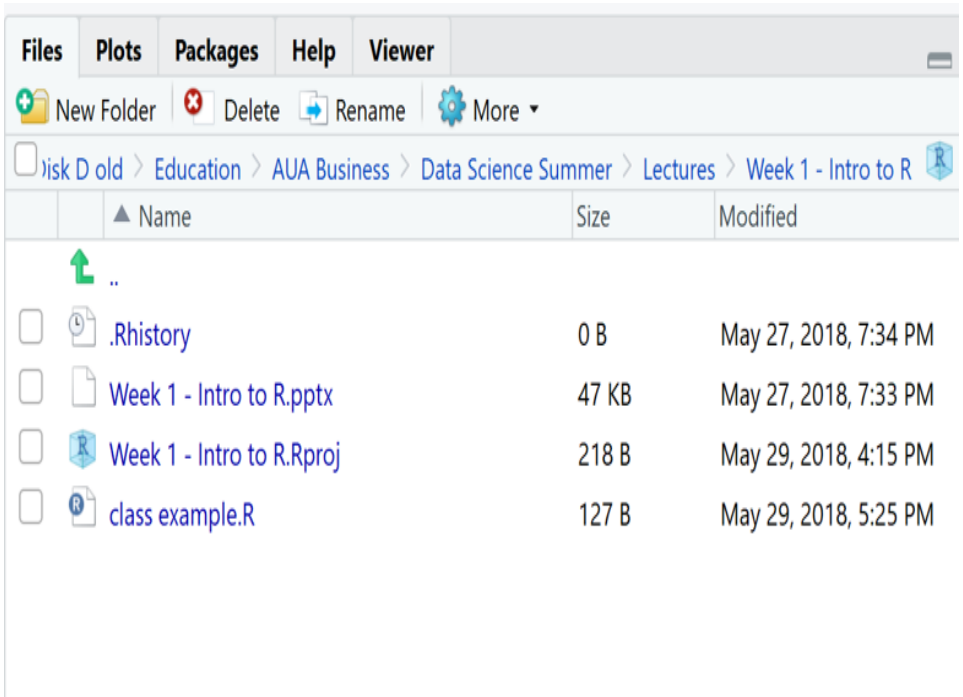
Intro to R

- The environment window contains objects (data, values, functions) R has currently stored in its memory.
- The history window shows all commands that were executed in the Console.



Intro to R

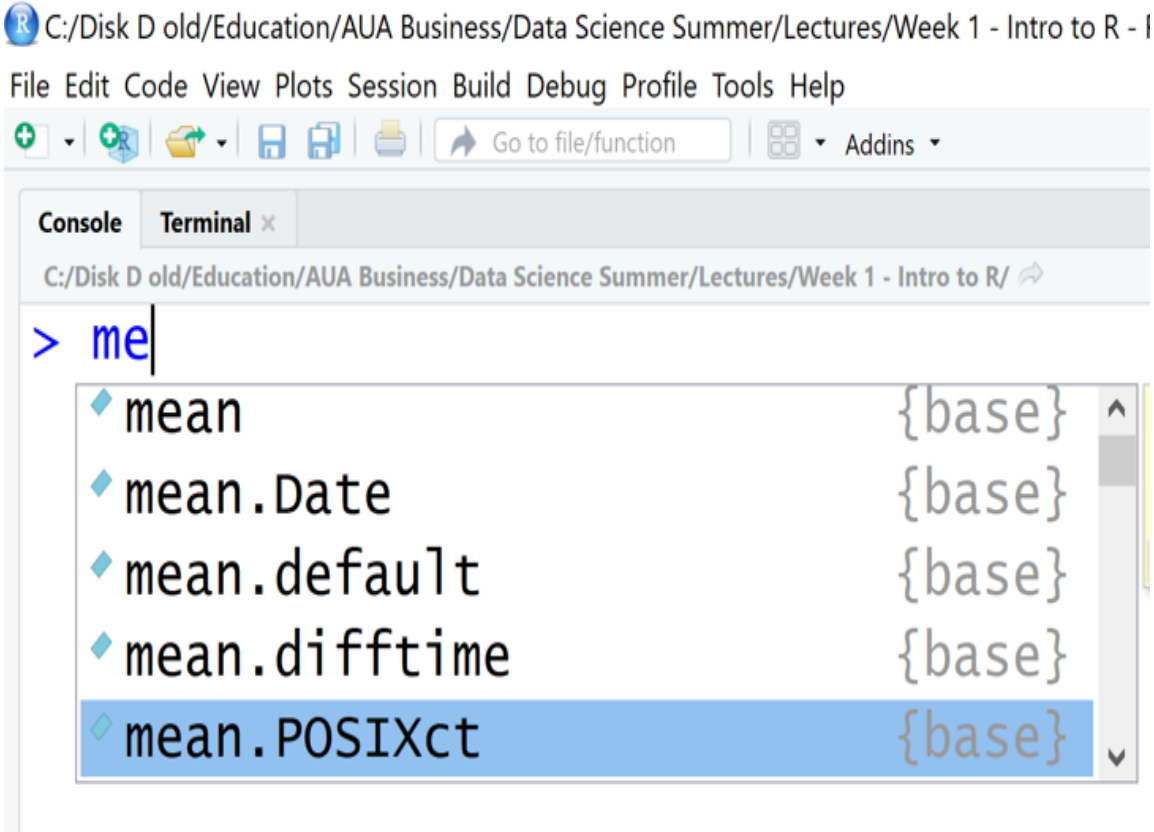
Bottom right: files, plots, packages, help, & viewer pane. Here you can open files, view plots, install and load packages, read main pages, and view markdown and other documents in the viewer tab.



Intro to R

Some useful shortcuts:

- type first few letters of the function/object then hit Tab to open dropdown menu with possible options

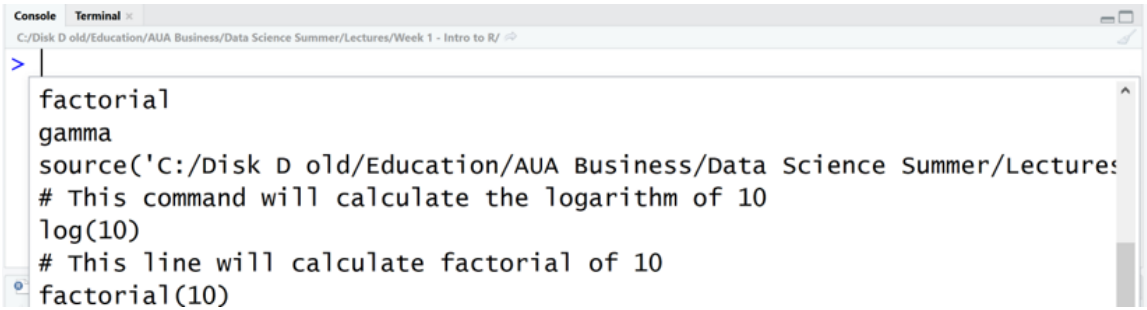


Intro to R

It's often the case that you want to re-execute commands that you previously entered. The RStudio console supports the ability to recall previous commands using the arrow keys:

- **Up** - Recall previous command(s)
- **Down** - Reverse of Up

You can even view a list of your recent commands by pressing **Ctrl+Up** on Windows or **Command+Up** on a Mac.



Intro to R

- When you download R from the CRAN, you get that "base" R system.
- The base R system comes with basic functionality; implements the R language.
- One reason R is so useful is the large collection of packages that extend the basic functionality of R.
- R packages are developed and published by the larger R community.

Intro to R

- Packages can be installed with the install.packages() function in R.
- To install a single package, pass the name of the library to the install.packages() function as the first argument.
- The following code installs the devtools package from CRAN.

```
install.packages("devtools")
```

Intro to R

- The package needs to be installed only once
- To load the package into R environment you need to use function library()
- You need to load the library everytime you start a new R

```
library(devtools)
```

Intro to R

- You can also install packages from github
- The following code will install the following package from github

```
library(devtools)
install_github("christophM/iml")
```

If you do not want to load the entire package but want to use some function from it, use the following command

package::function_name

```
devtools::install_github("christophM/iml")
```

Intro to R

To access help/documentation on a function from R base package

```
?mean
```

To access help/documentation on function from a library

```
#??geom_path
```

The same

```
#?ggplot2::geom_path
```

Help on the package

```
help(package='ggplot2')
```

Intro to R

- Each package on CRAN has its own webpage
- This includes documentation and sometimes includes vignettes

- If you have a specific task to do, then look at R Task View
- Here are all the packages and R functionality described for the Time series analysis

Intro to R programming language

- Anything in R is an object.
- Objects are assigned values using <- . (An equal sign = can also be used.) For e.g., the following command assigns value 5 to object x.

```
x <- 5
x
```

```
## [1] 5
```

- R is case sensitive, thus Data10 and data10 are two different objects.
- A tidy code requires a space before the assignment operator and a space after.
  - You can see what is inside the object just by simple entering the name of the object in command line
  - When the object is created it should appear in your Environment Window
  - If the object is not in your environment window you cannot work with it
  - The assignment operator works in the opposite direction as well

Intro to R programming language

A basic concept in (statistical) programming is called a variable.

A variable allows you to store a value (e.g. 4) or an object (e.g. a function description) in R. You can then later use this variable's name to easily access the value or the object that is stored within this variable.

You can assign a value 4 to a variable my_var with the command

```
my_var <-4
my_var
```

```
## [1] 4
```

Intro to R programming language

You can do basic operations with variables

```
my_var <- 4
your_var <- 5
my_var + your_var
```

```
## [1] 9
```

Or you can create a new variable

```
var2<- my_var + your_var
var2
```

```
## [1] 9
```

Intro to R programming language

R works with numerous data types.
Some of the most basic types to get started are:

- Decimals values like 4.5 are called numerics.
- Natural numbers like 4 are called integers. Integers are also numerics.
- Boolean values (TRUE or FALSE) are called logical.
- Text (or string) values are called characters.

Note how the quotation marks indicate that "some text" is a character.

Intro to R programming language

- Assign integer 42 to variable my_numeric
- String “universe“ to my_character
- **FALSE** to boolean my_logical

Intro to R programming language

Vector is a type of data structure that contains similar types of data, i.e., integer, double, logical, complex, etc. In order to create a vector c() function is used.

```
x <- c(10,5,6)
x
```

```
## [1] 10  5  6
```

```
x1 <- c(1:10)
x1
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

The same as (only when you have a sequence)

```
x1 <- 1:10
x1
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

Working with vectors

Working with vectors

Create 2 new vectors

```
revenue <- c(140, 50, 20, 120, 240)
expenses <- c(24, 50, 100, 50, 10)
```

```
names(revenue) <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
```

Working with vectors

Subseting vector by element name

```
revenue["Monday"]
```

```
## Monday
##    140
```

Working with vectors

Subseting element from vector by position. In R indexing starts from 1

```
revenue[4]
```

```
## Thursday
##      120
```

You can select 1 and 3 elements

```
revenue[c(1,3)]
```

```
##    Monday Wednesday
##       140        20
```

Or elements from 1 to 4

```
1:4
```

```
## [1] 1 2 3 4
```

```
revenue[c(1:4)]
```

```
##    Monday   Tuesday Wednesday  Thursday
##       140        50        20       120
```

Working with vectors

Arithmetic operations of vectors are performed member-by-member, i.e., memberwise.

For example, the following three statements are completely equivalent:

c(1, 2, 3) + c(4, 5, 6)

c(1 + 4, 2 + 5, 3 + 6)

c(5, 7, 9)

Working with vectors

Find daily income

```
inc_d <- revenue-expenses
inc_d
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##       116         0       -80        70       230
```

Find the days with positive revenue

This will give you a boolean vector

```
inc_d>0
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##      TRUE     FALSE     FALSE      TRUE      TRUE
```

subset

```
inc_pos<-inc_d[inc_d>0]
inc_pos
```

```
##    Monday  Thursday    Friday
##       116        70       230
```

Working with vectors

You can make comparison between vectors two

```
revenue > expenses
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##      TRUE     FALSE     FALSE      TRUE      TRUE
```

Working with vectors

The (logical) comparison operators known to R are:

- < for less than
- > for greater than
- <= for less than or equal to
- >= for greater than or equal to
- == for equal to each other
- != not equal to each other

Working with vectors

Recycling

```
u<-c(10,20,30)
v<-c(1,2,3,4,5,6,7,8,9)
u+v
```

```
## [1] 11 22 33 14 25 36 17 28 39
```

Working with vectors

You can use sum() command to get the sum of all element in the vector

```
sum(inc_d)
```

```
## [1] 336
```

Finding the average

```
mean(inc_d)
```

```
## [1] 67.2
```

You can do other operations with vectors too

```
inc_d*2
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##       232         0      -160       140       460
```

Working with vectors

In R, missing values are represented by the symbol NA (not available). Impossible values (e.g., dividing by zero) are represented by the symbol NaN (not a number).

```
d<-c(0,1,6,NA,5 ,NaN)
```

```
is.na(d)
```

```
## [1] FALSE FALSE FALSE  TRUE FALSE  TRUE
```

```
!is.na(d)
```

```
## [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE
```

Operations with matrices

In R, a matrix is a collection of elements of the same data type (numeric, character, or logical) arranged into a fixed number of rows and columns.
Since you are only working with rows and columns, a matrix is called two-dimensional.
You can construct a matrix in R with the matrix() function.
Consider the following example:

```
matrix(1:9, byrow = TRUE, nrow = 3)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

Operations with matrices

Box office Star Wars (in millions!)

```
new_hope <- c(460.998, 314.4)
empire_strikes <- c(290.475, 247.900)
return_jedi <- c(309.306, 165.8)
box_office <-c(new_hope, empire_strikes, return_jedi)
box_office
```

```
## [1] 460.998 314.400 290.475 247.900 309.306 165.800
```

Operations with matrices

Construct star_wars_matrix

```
star_wars_matrix <- matrix(box_office, nrow = 3, byrow = TRUE)
star_wars_matrix
```

```
##          [,1]  [,2]
## [1,] 460.998 314.4
## [2,] 290.475 247.9
## [3,] 309.306 165.8
```

Operations with matrices

We could have done the same using rbind command

```
star_wars_m1 <- rbind(new_hope, empire_strikes, return_jedi)
star_wars_m1
```

```
##                   [,1]  [,2]
## new_hope       460.998 314.4
## empire_strikes 290.475 247.9
## return_jedi    309.306 165.8
```

Operations with matrices

Naming rows and columns

```
region <- c("US", "non-US")
titles <- c("A New Hope", "The Empire Strikes Back", "Return of the Jedi")

# Name the columns with region
colnames(star_wars_matrix) <- region

# Name the rows with titles
rownames(star_wars_matrix) <- titles

# Print out star_wars_matrix
star_wars_matrix
```

```
##                             US non-US
## A New Hope                 460.998  314.4
## The Empire Strikes Back 290.475  247.9
## Return of the Jedi         309.306  165.8
```

Operations with matrices

Lets get worldwide box-office for the tree movies

```
worldwide_vector <- rowSums(star_wars_matrix)
worldwide_vector
```

```
##             A New Hope The Empire Strikes Back      Return of the Jedi
##                775.398                538.375                475.106
```

Get US and Non-US box office for all tree movies combined

```
combined_vector<-colSums(star_wars_matrix)
combined_vector
```

```
##       US   non-US
## 1060.779  728.100
```

Operations with matrices

Similar to vectors, you can use the square brackets [ ] to select one or multiple elements from a matrix. Whereas vectors have one dimension, matrices have two dimensions. You should therefore use a comma to separate that what to select from the rows from that what you want to select from the columns.

For example:

- my_matrix[1,2] selects the element at the first row and second column.
- my_matrix[1:3,2:4] results in a matrix with the data on the rows 1, 2, 3 and columns 2, 3, 4.

If you want to select all elements of a row or a column, no number is needed before or after the comma, respectively:

- my_matrix[,1] selects all elements of the first column.
- my_matrix[1,] selects all elements of the first row.

Operations with matrices

Take the first row - A new hope

```
new_hope<-star_wars_matrix[1,]
new_hope
```

```
##       US  non-US
## 460.998 314.400
```

Take the first column

```
US <- star_wars_matrix[,1]
US
```

```
##             A New Hope The Empire Strikes Back      Return of the Jedi
##                460.998                290.475                309.306
```

Operations with matrices

US box office for "A new hope"

```
star_wars_matrix[1,1]
```

```
## [1] 460.998
```

You can do the subseting by the names of rows and columns as well, if they are specified

```
star_wars_matrix[,"non-US"]
```

```
##               A New Hope The Empire Strikes Back      Return of the Jedi
##                    314.4                   247.9                   165.8
```

```
star_wars_matrix["A New Hope","non-US"]
```

```
## [1] 314.4
```

Dataframes

Dataframes

- Data frame is a data set that contain different data types instead of only one. You can combine numeric and character variables into the same dataset
- A data frame has the variables of a data set as columns and the observations as rows.
- Has the most use in Data Analytics

Working with dataframes

```
data(mtcars)
```

Look at the description

```
?mtcars
```

Working with dataframes

Look at the top 5 observations

```
head(mtcars, n=5)
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
```

Working with dataframes

Look at the bottom file observations

```
tail(mtcars, n=5)
```

```
##                 mpg cyl  disp  hp drat    wt qsec vs am gear carb
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.9  1  1    5    2
## Ford Pantera L 15.8   8 351.0 264 4.22 3.170 14.5  0  1    5    4
## Ferrari Dino   19.7   6 145.0 175 3.62 2.770 15.5  0  1    5    6
## Maserati Bora  15.0   8 301.0 335 3.54 3.570 14.6  0  1    5    8
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.6  1  1    4    2
```

Working with dataframes

```
str(mtcars)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

Working with dataframes

Look at the summary of the dataframe

```
summary(mtcars)
```

```
##       mpg             cyl             disp             hp       
##  Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
##  1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5  
##  Median :19.20   Median :6.000   Median :196.3   Median :123.0  
##  Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7  
##  3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0  
##  Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0  
##       drat             wt             qsec             vs        
##  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
##  1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000  
##  Median :3.695   Median :3.325   Median :17.71   Median :0.0000  
##  Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375  
##  3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000  
##  Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000  
##        am              gear            carb      
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000  
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000  
##  Median :0.0000   Median :4.000   Median :2.000  
##  Mean   :0.4062   Mean   :3.688   Mean   :2.812  
##  3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
```

Working with dataframes

Subseting the dataframes

Get the dimensions of the dataframe

```
dim(mtcars)
```

```
## [1] 32 11
```

Create a new dataframe with the first two columns and all the rows

```
df1 <- mtcars[,1:2]
dim(df1)
```

```
## [1] 32  2
```

Working with dataframes

Create a new dataframe with column 1,2,3 and 6, and first 10 rows

```
df2 <- mtcars[1:10,c(1:3,6)]
dim(df2)
```

```
## [1] 10  4
```

Create a new dataframe with all the variables, except 1 and 4

```
df3 <- mtcars[,-c(1,4)]
dim(df3)
```

```
## [1] 32  9
```

Working with dataframes

If the columns or rows have names, you can subset them by names as well

```
df4 <- mtcars[,c("mpg", "gear", "disp")]
names(df4)
```

```
## [1] "mpg"  "gear" "disp"
```

Working with dataframes

You can access a column in dataframe using $ sign if columns have names

```
mpg <- mtcars$mpg
mpg
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
## [16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
## [31] 15.0 21.4
```

Working with dataframes

You can subset a dataframe by rows by condition on any column

Keep only rows where mpg is more than or equal to 15

```
df5 <- mtcars[mtcars$mpg>=15, ]
summary(df5)
```

```
##       mpg             cyl             disp             hp       
##  Min.   :15.00   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
##  1st Qu.:17.55   1st Qu.:4.000   1st Qu.:120.2   1st Qu.: 94.0  
##  Median :21.00   Median :6.000   Median :160.0   Median :110.0  
##  Mean   :21.47   Mean   :5.852   Mean   :196.3   Mean   :131.6  
##  3rd Qu.:23.60   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:175.0  
##  Max.   :33.90   Max.   :8.000   Max.   :400.0   Max.   :335.0  
##       drat             wt             qsec             vs        
##  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
##  1st Qu.:3.115   1st Qu.:2.393   1st Qu.:16.96   1st Qu.:0.0000  
##  Median :3.770   Median :3.170   Median :18.00   Median :1.0000  
##  Mean   :3.666   Mean   :2.945   Mean   :18.03   Mean   :0.5185  
##  3rd Qu.:4.000   3rd Qu.:3.450   3rd Qu.:19.17   3rd Qu.:1.0000  
##  Max.   :4.930   Max.   :4.070   Max.   :22.90   Max.   :1.0000  
##        am              gear            carb      
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000  
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:1.500  
##  Median :0.0000   Median :4.000   Median :2.000
```

Working with dataframes

Can subset by rows using more than 1 condition.

AND logical operator

```
df6 <- mtcars[mtcars$mpg>15 & mtcars$carb<3,]
summary(df6)
```

```
##       mpg             cyl             disp             hp
##  Min.   :15.20   Min.   :4.000   Min.   : 71.1   Min.   : 52.0
##  1st Qu.:19.20   1st Qu.:4.000   1st Qu.: 95.1   1st Qu.: 66.0
##  Median :22.80   Median :4.000   Median :121.0   Median : 97.0
##  Mean   :23.61   Mean   :5.176   Mean   :177.7   Mean   :104.4
##  3rd Qu.:27.30   3rd Qu.:6.000   3rd Qu.:258.0   3rd Qu.:113.0
##  Max.   :33.90   Max.   :8.000   Max.   :400.0   Max.   :175.0
##       drat             wt             qsec             vs
##  Min.   :2.760   Min.   :1.513   Min.   :16.70   Min.   :0.0000
##  1st Qu.:3.150   1st Qu.:2.140   1st Qu.:17.05   1st Qu.:0.0000
##  Median :3.770   Median :2.780   Median :18.61   Median :1.0000
##  Mean   :3.692   Mean   :2.709   Mean   :18.73   Mean   :0.7059
##  3rd Qu.:4.080   3rd Qu.:3.435   3rd Qu.:19.90   3rd Qu.:1.0000
##  Max.   :4.930   Max.   :3.845   Max.   :22.90   Max.   :1.0000
##        am              gear            carb
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:1.000
##  Median :0.0000   Median :4.000   Median :2.000
```

Working with dataframes

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
!x	Not x
x | y	x OR y
x & y	x AND y

Working with dataframes

The OR logical operator

```
df7 <- mtcars[mtcars$mpg>15 | mtcars$carb<3, ]
summary(df7)
```

```
##       mpg             cyl             disp             hp       
##  Min.   :15.20   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
##  1st Qu.:17.88   1st Qu.:4.000   1st Qu.:120.2   1st Qu.: 93.5  
##  Median :21.00   Median :6.000   Median :160.0   Median :110.0  
##  Mean   :21.72   Mean   :5.769   Mean   :192.3   Mean   :123.8  
##  3rd Qu.:24.00   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:168.8  
##  Max.   :33.90   Max.   :8.000   Max.   :400.0   Max.   :264.0  
##       drat             wt             qsec             vs        
##  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
##  1st Qu.:3.098   1st Qu.:2.356   1st Qu.:17.02   1st Qu.:0.0000  
##  Median :3.810   Median :3.160   Median :18.15   Median :1.0000  
##  Mean   :3.671   Mean   :2.921   Mean   :18.16   Mean   :0.5385  
##  3rd Qu.:4.040   3rd Qu.:3.440   3rd Qu.:19.30   3rd Qu.:1.0000  
##  Max.   :4.930   Max.   :4.070   Max.   :22.90   Max.   :1.0000  
##        am              gear            carb      
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000  
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:1.250  
##  Median :0.0000   Median :4.000   Median :2.000  
##  Mean   :0.4615   Mean   :3.769   Mean   :2.385
```

Working with dataframes

Creating the final season schedule for Serie A.

The following varaibles describe the first 5 places for Serie A at 1996.

```
Team<-c("AC Milan","Juventus","Lazio","ACF Fiorentina","AS Roma","Parma AC")
Wins<-c(21,19,17,17,16,16)
Draws<-c(10,8,8,8,10,10)
Points<-c(73,65,59,59,58,58)
```

Working with dataframes

We can combine these variables into a dataframe

```
seriea <- data.frame(Team,Points, Wins, Draws)
str(seriea)
```

```
## 'data.frame':    6 obs. of  4 variables:
##  $ Team  : chr  "AC Milan" "Juventus" "Lazio" "ACF Fiorentina" ...
##  $ Points: num  73 65 59 59 58 58
##  $ Wins  : num  21 19 17 17 16 16
##  $ Draws : num  10 8 8 8 10 10
```

Working with dataframes

Working with variables

```
mean(seriea$Points)
```

```
## [1] 62
```

```
mean(seriea[,"Points"])
```

```
## [1] 62
```

Working with dataframes

We can create a new variable within the dataframe that will show the number of games each team has lost (total 34 games were played)

```
seriea$Lost<-34-(seriea$Wins+seriea$Draws)
```

Working with dataframes

Dataframe has column and rownames

```
colnames(seriea)
```

```
## [1] "Team"   "Points" "Wins"   "Draws"  "Lost"
```

```
rownames(seriea)
```

```
## [1] "1" "2" "3" "4" "5" "6"
```

Lists

Lists

- Vectors (one dimensional array): can hold numeric, character or logical values. The elements in a vector all have the same data type.
- Matrices (two dimensional array): can hold numeric, character or logical values. The elements in a matrix all have the same data type.
- Data frames (two-dimensional objects): can hold numeric, character or logical values. Within a column all elements have the same data type, but different columns can be of different data type.

Lists

```
# Vector with numerics from 1 up to 10
my_vector <- 1:10

# Matrix with numerics from 1 up to 9
my_matrix <- matrix(1:9, ncol = 3)

# First 5 elements of the built-in data frame mtcars
my_df <- mtcars[1:5,]

# a character
my_char<-"DataMata"

# combine into a list
my_list<-list(my_vector, my_matrix, my_df, my_char)
my_list
```

```
## [[1]]
##  [1]  1  2  3  4  5  6  7  8  9 10
##
## [[2]]
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

Lists

New list for movie shining

```
moviename<-"The Shining"
actors<-c("Jack Nicholson","Shelley Duvall",
          "Danny Lloyd","Scatman Crothers",
          "Barry Nelson" )
reviews<-data.frame(scores=c(4.5,4,5),
        sources=c("IMDb1", "IMDb2", "IMDb3"),
        comments =c(
          "Best Horror Film I Have Ever Seen",
          "A truly brilliant and scary film from Stanley Kubrick",
          "A masterpiece of psychological horror"))
```

Lists

Named list

```
shining_list<-list(moviename=moviename, actors=actors, reviews=reviews)
shining_list
```

```
## $moviename
## [1] "The Shining"
##
## $actors
## [1] "Jack Nicholson"   "Shelley Duvall"   "Danny Lloyd"      "Scatman Crothers"
## [5] "Barry Nelson"
##
## $reviews
##   scores sources                                                comments
## 1    4.5   IMDb1                          Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                     A masterpiece of psychological horror
```

Subsetting the list.

One way to select a component is using the numbered position of that component. For example, to "grab" the first component of shining_list you type

```
shining_list[[1]]
```

Lists

A quick way to check this out is typing it in the console. Important to remember: to select elements from vectors, you use single square brackets: [ ]. Don't mix them up!

You can also refer to the names of the components, with [[ ]] or with the $ sign. Both will select the data frame representing the reviews:

```
shining_list[["reviews"]]
```

```
##   scores sources                                                comments
## 1    4.5   IMDb1                          Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                     A masterpiece of psychological horror
```

```
shining_list$reviews
```

```
##   scores sources                                                comments
## 1    4.5   IMDb1                          Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                     A masterpiece of psychological horror
```

Lists

Besides selecting components, you often need to select specific elements out of these components. For example, with shining_list[[2]][1] you select from the second component, actors (shining_list[[2]]), the first element ([1])

```
shining_list[[2]][1]
```

```
## [1] "Jack Nicholson"
```

Lists

Adding new value to the list, simply use c

```
shining_list<-c(shining_list, year=1980)
shining_list
```

```
## $moviename
## [1] "The Shining"
##
## $actors
## [1] "Jack Nicholson"   "Shelley Duvall"   "Danny Lloyd"      "Scatman Crothers"
## [5] "Barry Nelson"
##
## $reviews
##   scores sources                                                comments
## 1    4.5   IMDb1                          Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                    A masterpiece of psychological horror
##
## $year
## [1] 1980
```

Functions

Functions

- In programming, you use functions to incorporate sets of instructions that you want to use repeatedly or that, because of their complexity, are better self-contained in a sub program and called when needed.
- A function is a piece of code written to carry out a specified task; it can or can not accept arguments or parameters and it can or can not return one or more values.
- In fact, there are several possible formal definitions of 'function' spanning from mathematics to computer science.
- **Generically, its arguments constitute the input and return values as output.**

Functions

- Functions have named arguments which potentially have default values.
- The formal arguments are the arguments included in the function definition.
- The formals function returns a list of all the formal arguments of a function.
- Not every function call in R makes use of all the formal arguments.
- Function arguments can be missing or might have default values.

Functions

The function runif() generates random numbers from continuous uniform distribution

Arguments are:

- n, number of observations, with no default - so needs to be specified
- min, the minimum
- max, the maximum

```
formals(runif)
```

```
## $n
##
##
## $min
## [1] 0
##
## $max
## [1] 1
```

```
args(runif)
```

```
## function (n, min = 0, max = 1)
## NULL
```

Functions

R functions' arguments can be matched **positionally** or **by name**. So the following calls to runif() are all equivalent (you can mix them as well).

- Matching by name

```
runif(min=1, n=10, max=2)
```

```
##  [1] 1.406951 1.822841 1.603863 1.860815 1.252198 1.172146 1.885811 1.665590
##  [9] 1.261623 1.472472
```

- Matching by position

```
runif(10,1,2)
```

```
##  [1] 1.017970 1.043103 1.623214 1.767684 1.724066 1.961791 1.684693 1.064055
##  [9] 1.258395 1.139368
```

Note, as the process is random you might get different results.

Functions

Matched by position, this will give an error

```
runif(1,10,2)
```

```
## Warning in runif(1, 10, 2): NAs produced
```

```
## [1] NaN
```

Functions

- If the argument has a default value and is not defined then the default value is used.
- Generated 10 numbers with min=0, max=1

```
runif(10)
```

```
##  [1] 0.8215719 0.4998325 0.4965346 0.5532002 0.7513662 0.8606219 0.3523443
##  [8] 0.8498498 0.8944144 0.2612416
```

Functions

You can look at the arguments of the function and their description by hitting **Tab** inside the brackets

